Towards Integrating Microservices with Adaptable Enterprise Architecture

Justus Bogner
Hewlett Packard Enterprise
Herman Hollerith Center
Boeblingen, Germany
justus.bogner@hpe.com

Alfred Zimmermann
Reutlingen University
Herman Hollerith Center
Boeblingen, Germany
alfred.zimmermann@reutlingen-university.de

***Abstract*—IT environments that consist of a very large number of rather small structures like microservices, Internet of Things (IoT) components, or mobility systems are emerging to support flexible and agile products and services in the age of digital transformation. Biological metaphors of living and adaptable ecosystems with service-oriented enterprise architectures provide the foundation for self-optimizing, resilient run-time environments and distributed information systems. We are extending Enterprise Architecture (EA) methodologies and models that cover a high degree of heterogeneity and distribution to support the digital transformation and related information systems with micro-granular architectures. Our aim is to support flexibility and agile transformation for both IT and business capabilities within adaptable digital enterprise architectures. The present research paper investigates mechanisms for integrating Microservice Architectures (MSA) by extending original enterprise architecture reference models with elements for more flexible architectural metamodels and EA-mini-descriptions.**

Keywords—Microservices; Adaptable Enterprise Architecture; Enterprise Architecture Management;

I. Introduction

The fast moving process of digitization [1] demands flexibility in order to adapt to rapidly changing business requirements and newly emerging business opportunities. New features have to be developed and deployed to the production environment a lot faster. To be able to cope with this increased velocity and pressure, a lot of software developing companies have switched to integrate Microservice Architectures (MSA). Applications built this way consist of several fine-grained services that are independently scalable and deployable. This enables teams to choose the best-fitting technologies for these services and organize them around well-defined business capabilities. The responsibility for all development and operation activities of such a microservice is then assigned to one team. Using MSAs, organizations can expand agility and flexibility for business and IT systems, which fits better to small sized integrated systems and is vital in the age of digital transformation.

However, the technological and business architectural impacts of microservices based applications directly affect their composition into the digital enterprise architecture and their related systems. Enterprise Architecture Management [2] for Services Computing is usually the approach of choice to organize, manage, and utilize distributed IT and business capabilities. These capabilities consist of a wide variety of systems and services including Microservice Architectures, Internet of Things based applications, larger Web and REST Services, Cloud Computing environments and Big Data solutions. It is difficult to manage the extreme distribution, diversity and volatility of micro-granular systems and services with a traditional EA approach.

Our current paper focuses on the following research questions:

RQ1: What are architectural properties of microservices and what are the resulting implications for Enterprise Architecture Management?

RQ2: What is a feasible Enterprise Architecture integration approach for merging model structures of multiple cooperating microservices?

RQ3: What are the results and implications of integrating microservices into an adaptable Enterprise Architecture?

The following section 2 sets the fundamental context for application architecture with a microservice approach. Section 3 describes our research platform for digital enterprise architecture, which was extended by concepts from architectural adaptation mechanisms and a specific metamodel integration method while section 4 presents exemplary results of such an EA integration based on microservices. Finally in section 5, we summarize our research findings and limitations and present our ongoing work in academic and practical environments as well as our future research plans.

II. Microservices: Fundamentals and Architecture

The term microservices became popular around 2013 and refers to a fine-grained style of service-oriented architecture (SOA) applications combined with several DevOps elements. James Lewis and Martin Fowler define a Microservice Architecture [3] as an approach for building a single application as a set of small independent services. Each of these services runs in its own process and communicates with lightweight mechanisms like an HTTP resource API. Microservices are built around business capabilities and are independently deployable by a fully automated deployment machinery. Typically, there is only very little centralized management for these services. Microservices may be written

978-1-4673-9933-3/16/$31.00 ©2016 IEEE

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.

in different programming languages and can use different data storage technologies. As opposed to big monolithic applications, a single microservice tries to represent a unit of functionality that is as small and coherent as possible. This unit of functionality or business capability is often referred to as a *bounded context*, a term that originates from Domain-Driven Design (DDD) [3]. A benefit that comes with microservices is the possibility to use the best-fitting technology for each *bounded context*. Other advantages include increased application resilience (if one microservice fails, the others may not be affected, at least if there is no chaining) as well as independent and efficient scalability instead of replicating the complete monolith, and faster and easier deployment [4]. Especially the last advantage is an important step towards the desired agility of business and IT systems.

However, microservices also come with the need for a strong DevOps culture [5] to handle the increased distribution level and deployment frequency. Moreover, while each single microservice may be of reasonably low complexity compared to a monolithic application, the overall complexity of the system has not been reduced at all. Gary Olliffe, a research director at Gartner, distinguishes between the inner architecture and the outer architecture of microservices (see Fig. 1, [6]).

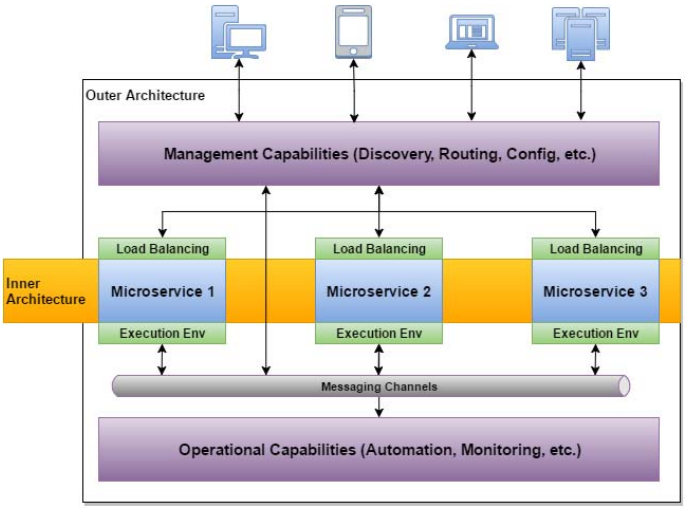

Fig. 1. Microservices Inner and Outer Architecture, based on [6]

By splitting up a big monolith into more fine-grained independent services, you shift most of the hindering complexity from the inner architecture to the outer architecture, where inter-service communication, service discovery, or operational capabilities have to be managed.

While enabling technological heterogeneity is usually considered an advantage of MSA [4] that allows the selection of the best tool for the job, reduces the possibility of lock-ins for outdated technology, and supports a culture of innovation and experimentation. However, MSA also comes with some risks for the organization. An explosion of technological diversity can quickly become overwhelming and unmanageable. Moreover, you are dependent on employees with the corresponding skills to handle these technologies and programming languages.

This is why most organizations that use MSAs either provide some very basic standardization without limiting their teams' choices too much or encourage the use of only a certain technology subset by offering comfortable tooling and infrastructure support for selected languages. Both approaches work reasonably well and prevent the existence of e.g. three different versions of Java or the use of six different web servers. This difficulty of keeping a healthy amount of governance and standardization while still allowing enough technological heterogeneity to not hinder innovation and agility can be addressed by Enterprise Architecture Management. However, classical approaches to EAM are often not flexible enough for the kind of diversity and distribution present in MSAs.

III. Microservices in the Context of EAM

Today, Enterprise Architecture Management [7]–[9] defines quite a large set of different views and perspectives with frameworks, standards, tools, and practical expertise. We argue in this paper that a newly refocused digital enterprise architecture approach should support digitization of products and services and should be both holistic [2], [10] and easily adaptable [11] to support the digital transformation with new business models and technologies that are based on a large number of microstructured digitization systems with own micro-granular architectures like IoT, mobility devices or – as presented in this paper – with microservices.

Enterprise Services Architecture Reference Cube (ESARC) [10] is our architectural reference model for an extended view on evolved digital enterprise architectures (see Fig. 2). It is more specific than existing architectural standards of EAM [12], [13] and extends these standards for services and cloud computing. ESARC provides a holistic classification model with eight integral architectural domains. While it is applicable for concrete architectural instantiations to support digital transformations, it still abstracts from a concrete business scenario or technologies. The Open Group Architecture Framework [12] provides the basic blueprint and structure for our extended service-oriented enterprise architecture domains.

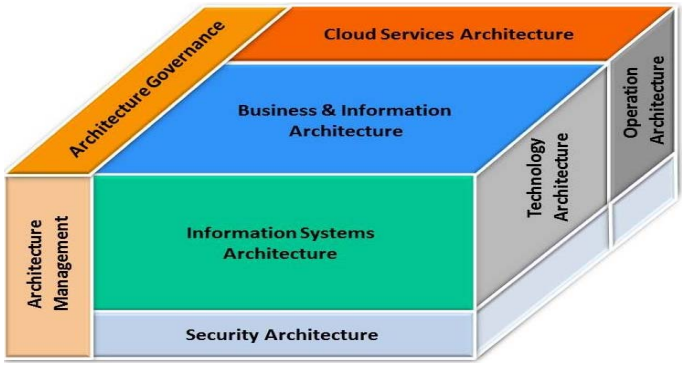

Fig. 2. Enterprise Services Architecture Reference Cube [2], [10]

Most aspects of microservices are located within the Information Systems Architecture (ISA, green) and the Technology Architecture (TA, light grey) areas. Since ISA is essentially service-based (see Fig. 3), it includes a lot of fitting

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.

elements for microservice implementations, e.g. *Utility Services*, *Task Services*, or *Rule Services*. While web services or larger REST services in the context of "traditional" SOA are the technologies that are normally used for implementing these capabilities, microservices present an interesting alternative or extension approach. At least the functional service classification within the ISA model is mostly up to the task of covering the heterogeneity of microservices. Revising and adjusting this domain will still be of value in the near future though.

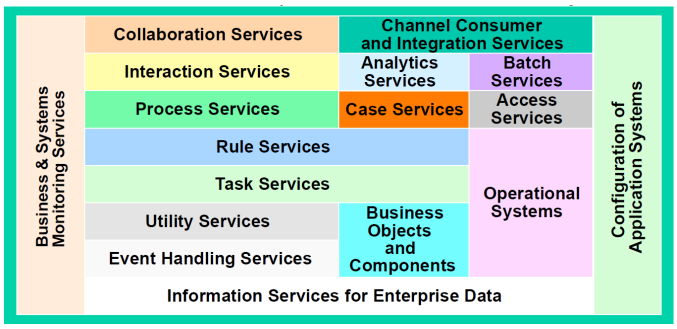

Fig. 3. ESARC – Information Systems Architecture [10]

Within the Technology Architecture (TA) of ESARC (see Fig. 4) however, the placement and embedding of microservices is a bit more complicated. A lot of elements like the *Enterprise Service Bus*, *Reliable Messaging*, or the *Process Orchestration Server* are very focused on large and intricate SOA environments. Systems based on MSAs on the other hand use lightweight messaging or HTTP as communication mechanisms, prefer choreography over orchestration, and are also not very well suited for complex and long-running *Transactions* with data updates, as they are usually embracing eventual consistency within a highly distributed environment. [4] So in order to have a more fitting representation of microservices in this architectural reference model, some adjustments seem to be necessary, either by generalizing and extending the existing model or by creating a new specialized version of ESARC for a microservices context. However, such model adjustments are definitely not enough to reach a suitable amount of management, control and governance within application environments that consist of a lot of MSAs. This is why chapter IV presents a new approach to make EAM more adaptable and suited for heterogeneity and unpredictability.

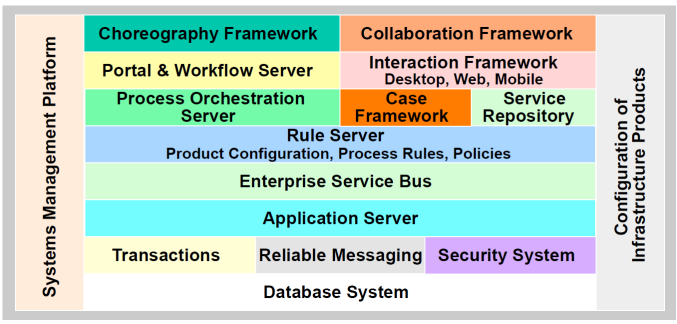

Fig. 4. ESARC – Technology Architecture [10]

IV. Adaptable Enterprise Architecture

Volatile technologies, requirements, and markets typically drive the evolution of business and IT services. Adaptation is a key success factor for the survival of digital enterprise architectures [14], platforms, and application environments. Tiwana presents the idea of digital *ecosystems* in [15] that can be linked with main strategic drivers for system development and system evolution. Reacting rapidly to new technology and market contexts improves the fitness of such adaptive ecosystems. Being a bit closer to the architecture and design of systems, Breu et al coined the *Living Models* paradigm that is concerned with the model based creation and management of dynamically evolving systems [16], [17]. An architectural style with a very similar intention is evaluated by Ferreira in [18]: Adaptive Object-Modelling and its patterns and usage provide useful techniques to react to changing user requirements, even during the runtime of a system. Moreover, Roth et al cover model conflict resolution of automated EA documentation [19] and summarize integration foundations for federated EA model management [20]. Finally, we have to consider internal factors: The alignment of Architecture Governance [7], [21] shapes resiliency, scalability, and reusability of components and services for distributed information systems.

Adaptability in the context of EA and microservices is mostly concerned with heterogeneity, distribution, and volatility. To integrate a huge amount of dynamically growing architectural descriptions of very different microservices (or any other type of "microstructure" with micro-granular architecture) into a consistent enterprise architecture is a considerable challenge. Unfortunately, no solution approaches for microservices within EA were present in research literature at the time of publication. In order to address this problem, we are currently working on formalizing small-decentralized mini-metamodels, models, and data of architectural microservice descriptions (EA-Mini-Descriptions). EA-Mini-Descriptions consist of partial EA-Data, partial EA-Models, and partial EA-Metamodels associated with microservices. They are based on the Meta Object Facility (MOF) standard [22] of the Object Management Group (OMG). MOF describes a four layer architecture that can be used as a Domain Specific Language (DSL) to create metamodels. The highest layer M3 represents the abstract language concepts used in the lower M2 layer and is therefore the meta-metamodel layer. The next layer M2 is the metamodel layer and defines the language entities for M1 (e.g. constructs from UML [23], ArchiMate [13], or OWL [24]). Instantiations of these languages then form the layer M1 that contains models in the specified language. These models are a structured representation of the lowest layer M0 that is formed by collected concrete data from real-world use cases.

For building our EA-Mini-Descriptions, we applied the four layers of MOF [22] to provide sufficient information structures for an EA integration scenario with microservices (see Fig. 5). M0 and M1 are local layers to a single microservice (*cell* metaphor). While M0 consists of operational run-time or monitoring data, M1 contains important meta-data of the microservice (e.g. purpose, API endpoints, or usage costs) as well as its inner architectural model (e.g. components or communication channels). On top of these, the layer M2 acts as a global meta-model layer that holds necessary information

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.

for several collaborating microservices (*body* metaphor, combining several *cells*). It incorporates architectural MSA-specific meta-models and ontologies while also providing the important integration rules for the semi-automatic integration of specific metamodels to the overall integrated and dynamically growing EA metamodel from the composition of EA-Mini-Descriptions. On top of that, M3 specifies the languages and semantic representations that we are using for modeling and representing these adaptable enterprise architecture metamodels.

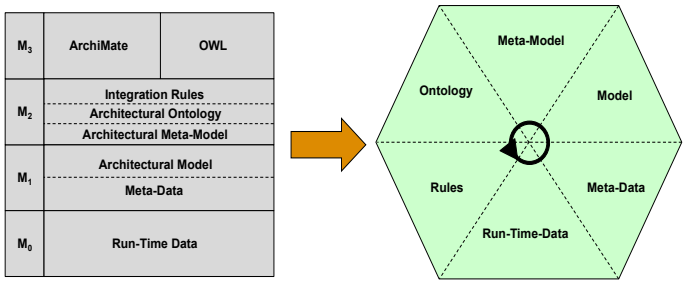

Fig. 5. Structure of EA-Mini-Descriptions

The main question of our research in progress asks, how we can dynamically federate these EA-Mini-Descriptions to a global and highly scalable EA model and information base. In order to exemplarily illustrate the nature of our integration approach in practice, chapter V presents several concrete microservices and their integration procedure.

V. Integration Example

Let's consider the example of a microservices based web shop application. Such an application will consist of a variety of different microservices that are responsible for different parts of functionality, but may have some interaction or relations. They may be implemented with very different technologies and their internal architecture may differ substantially. While there are certainly many more in such a scenario, we will focus on three concrete microservices for this example, an *OrderService*, a *BillingService*, and a *ShippingService*. In Fig. 6, their EA-Mini-Descriptions are shown before the integration approach. They have some interactions and exchange data like order IDs or credit card details, but they are separate architecture descriptions. In order to reach a holistic EA view for the web shop company, these microservices and their architectural metamodels have to be integrated by creating or extending an existing architecture reference model.

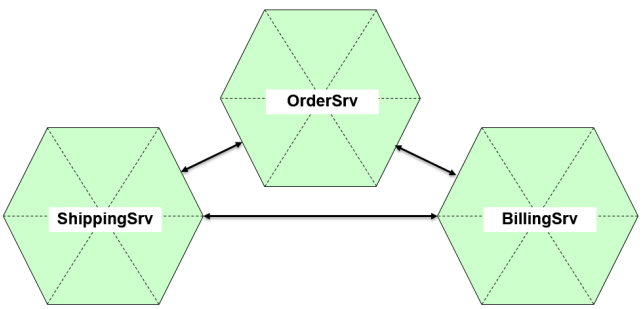

Fig. 6. Web Shop Example: Microservices Pre-Integration

This is why we are extending our previous architecture evolution approach to combine and adapt valuable parts of existing EA frameworks and metamodels from theory and practice. The Enterprise Services Architecture Model Integration (ESAMI) [10] method is based on correlation analysis, which provides an instrument for a systematic manual integration process. Typically, this process of pair wise mappings is of quadratic complexity. We have linearized the complexity of these architectural mappings by introducing a neutral and dynamically extendable architectural reference model, which is supplied and dynamically extended from previous mapping iterations. Furthermore, we have adopted modeling concepts from ISO/IEC 42010 [25], like *Architecture Description*, *Viewpoint*, *View*, and *Model*. These architectural metamodels are composed of their elements and relationships and are represented by architecture diagrams. The ESAMI approach is based on special correlation matrices (see Fig. 7), which are handled by a manual process to identify similarities between analyzed model elements. The chosen elements are then integrated according to their most valuable contribution towards a holistic reference model. In each iteration of this bottom-up approach, we are analyzing the fit of each new microservice metamodel in comparison with the context of the existing integrated set of services' metamodels. The *Correlation Index* for the different microservices (red middle columns) with respect to the current *Reference Model* (yellow columns on the left) is created. Based on these *Correlation Indices*, the *Integration Options* for each service (green columns on the right) are chosen and the selection is integrated into the *Reference Model*. This continuous model refinement allows to integrate even extremely heterogeneous microservices that may not even share a complete metamodel.

Reference Model			Correlation Index			Integration Options		
Viewpoint	Model	Element	OrderSrv	ShippingSrv	BillingSrv	OrderSrv	ShippingSrv	BillingSrv
Business Actor	Customer	CustomerID	2	1	1	m	p	p
		Name	3	0	2	m	r	p
		Address	0	2	1	r	p	p
		Payment						r
		...						...
Passive Structure	Product	ProdID						p
		ProdName						m
		ProdDescr						m
		ProdComp						r
		Rate	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

0 no correlation
1 low correlation
2 medium correlation
3 strong correlation

r reject
p partially
m mandatory (leading model)

Fig. 7. Correlation Analysis and Integration Matrices

In addition to these analysis, synthesis, and consolidation activities, ESAMI also includes steps for developing the ontology of the architectural reference model. Ontologies [26], [27] provide a fundamental base for thorough understanding of the integrated architectural concepts as well as for additional knowledge representation and inference mechanisms. They represent a common vocabulary for enterprise architects, who need to share their information based on explicitly defined concepts. Together with the *Reference Model*, they can be a valuable asset for supporting the integration with additional semantics to combine several seemingly stand-alone microservices (*cells*) into a larger collaborating construct (*body*). A high-level example of this bottom-up integration approach can be seen in Fig. 8. Moreover, ontologies include

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.

the ability to infer transitive knowledge for automatically deciding the integration options for partially known integrations of micro-granular architectures into an overall EA metamodel.

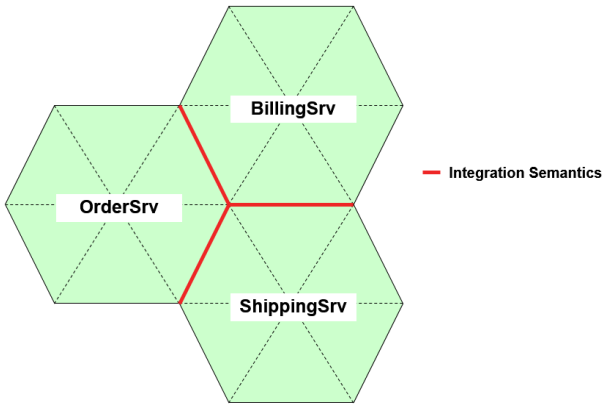

Fig. 8. Web Shop Example: Microservices Post-Integration

By integrating micro-granular architectural cells in this way, the integrated overall architectural metamodel becomes adaptable and can be automatically synthesized by considering the integration context from a growing number of previous integrations. Only in case of unpredictable integrations of completely new micro-granular architecture metamodels, we have to consider manual support by traditional human based architectural decisions.

VI. Conclusion

In this paper, we presented architectural properties of microservices and identified the ensuing need for advanced EA methodologies for the integration of large numbers of heterogeneous and frequently changing structures with a micro-granular architecture into an overall adaptable EA architecture. According to our research questions, we have leveraged a new model of extended Enterprise Architecture, which is well suited for integrating adaptive models and transformation mechanisms. Our EA-Mini-Descriptions can serve as flexible metamodels for microservices that can be combined into larger entities. Through a manual correlation-based model analysis and integration approach, we presented means to merge these microservices into a holistic, but dynamically adjusting reference architecture. Enterprises can use this to systematically manage microservices in practice, which are supported by an integrated multi-dimensional EA. However, this research is still ongoing. So far, we have neither performed larger prototypical studies with this approach nor a systematic evaluation and validation. The main purpose of this paper is to point out the deficiencies concerning adaptability when dealing with MSAs in classical EAM approaches, to present an alternative approach based on ESAMI, and to stimulate a discussion about its feasibility and potential alternative methods.

Future research may include a large integration prototype experiment and a thorough validation of the results in some practical use cases. Moreover, the automatic machine-supported creation of our EA-Mini-Descriptions (at least partially) will be essential for a wide adoption of such a methodology. Similarly, it may be of interest to support the manual integration decision by automated systems, e.g. via mathematical comparisons (similarity, Euclidean distance), semantic integration rules, or data analytics and data mining techniques. While such procedures cannot fully eliminate the necessity for human involvement, they should significantly ease associated manual efforts and reduce the rate of architectural errors in traditional EA models and approaches.

References

[1] G. Westerman, D. Bonnet, and A. McAfee, *Leading Digital: Turning Technology Into Business Transformation*. Harvard Business Press, 2014.

[2] A. Zimmermann, H. Buckow, H.-J. Groß, O. F. Nandico, G. Piller, and K. Prott, “Capability Diagnostics of Enterprise Service Architectures Using a Dedicated Software Architecture Reference Model,” in *2011 IEEE International Conference on Services Computing*, 2011, pp. 592–599.

[3] E. Evans, *Domain-driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.

[4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 1st ed. O’Reilly Media, 2015.

[5] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect’s Perspective*, 1st ed. Addison-Wesley Professional, 2015.

[6] G. Olliffe, “Microservices: Building Services with the Guts on the Outside,” 2015. [Online]. Available: http://blogs.gartner.com/gary-olliffe/2015/01/30/microservices-guts-on-the-outside/. [Accessed: 18-Mar-2016].

[7] M. Lankhorst, *Enterprise Architecture at Work*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.

[8] P. Johnson, R. Lagerström, M. Ekstedt, and M. Österlind, “IT Management with Enterprise Architecture,” *KTH, Stock.*, 2014.

[9] B. Beshilas, “Collaborative Enterprise Architecture by Stefan Bente, Uwe Bombosch, and Shailendra Langade,” *SIGSOFT Softw. Eng. Notes*, vol. 38, no. 1, p. 54, 2013.

[10] A. Zimmermann, K. Sandkuhl, M. Pretz, M. Falkenthal, D. Jugel, and M. Wissotzki, “Towards an integrated service-oriented reference enterprise architecture,” *Proc. 2013 Int. Work. Ecosyst. Archit. - WEA 2013*, pp. 26–30, 2013.

[11] A. Zimmermann, B. Gonen, R. Schmidt, E. El-Sheikh, S. Bagui, and N. Wilde, “Adaptable Enterprise Architectures for Software Evolution of SmartLife Ecosystems,” in *2014 IEEE 18th International Enterprise Distributed Object Computing Conference Workshops and Demonstrations*, 2014, pp. 316–323.

[12] V. Haren, *TOGAF Version 9.1*, 10th ed. Van Haren Publishing, 2011.

[13] T. O. Group, *ArchiMate{®} 2.0 Specification*. van Haren Publishing, 2012.

[14] A. Zimmermann, R. Schmidt, K. Sandkuhl, M. Wissotzki, D. Jugel, and M. Mohring, “Digital Enterprise Architecture - Transformation for the Internet of Things,” *2015 IEEE 19th Int. Enterp. Distrib. Object Comput. Work.*, no. October, pp. 130–138, 2015.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.

[15] A. Tiwana, *Platform Ecosystems: Aligning Architecture, Governance, and Strategy*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2014.

[16] R. Breu, B. Agreiter, M. Farwick, M. Felderer, M. Hafner, and F. Innerhofer-Oberperfler, "Living Models - Ten Principles for Change-Driven Software Engineering," *Int. J. Softw. Informatics*, no. 231101, 2011.

[17] T. Trojer, M. Farwick, M. Häusler, and R. Breu, "Living Modeling of IT Architectures: Challenges and Solutions," in *Software, Services, and Systems: Essays Dedicated to Martin Wirsing on the Occasion of His Retirement from the Chair of Programming and Software Engineering*, R. Nicola and R. Hennicker, Eds. Cham: Springer International Publishing, 2015, pp. 458–474.

[18] H. S. Ferreira, "Adaptive Object-Modelling: Patterns, Tools and Applications," University of Porto, Faculty of Engineering, 2010.

[19] S. Roth, M. Hauder, F. Michel, D. Münch, and F. Matthes, "Facilitating conflict resolution of models for automated enterprise architecture documentation," *19th Am. Conf. Inf. Syst. AMCIS 2013 - Hyperconnected World Anything, Anywhere, Anytime*, vol. 3, pp. 1662–1672, 2013.

[20] S. Roth, "Federated Enterprise Architecture Model Management -- Conceptual Foundations, Collaborative Model Integration, and Software Support," 2014.

[21] J. W. Ross, P. Weill, and D. Robertson, *Enterprise Architecture as Strategy: Creating a Foundation for Business Execution*. Harvard Business School Press, 2006.

[22] OMG, "OMG Meta Object Facility (MOF) Core Specification, Version 2.5," no. August. Jun-2011.

[23] OMG, "OMG Unified Modeling Language (OMG UML), Version 2.5," *InformatikSpektrum*, vol. 21, no. May. p. 758, Mar-2010.

[24] W3C, "OWL 2 Web Ontology Language. Structural Specification and Functional-Style Syntax." 2009.

[25] D. Emery and R. Hilliard, "Every Architecture Description Needs a Framework : Expressing Architecture Frameworks Using ISO / IEC 42010," in *WICSA/ECSA*, 2009, pp. 31–40.

[26] The Open Group, "Open Group Standard Service-Oriented Architecture Ontology." The Open Group, 2014.

[27] A. Zimmermann and G. Zimmermann, "Enterprise Architecture Ontology for Services Computing," in *SERVICE COMPUTATION 2012 : The Fourth International Conferences on Advanced Service Computing*, 2012, no. c, pp. 64–69.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:26:18 UTC from IEEE Xplore. Restrictions apply.